

EXPERIMENT 8

Design and Train a Convolutional Neural Network (CNN)

1.Dataset Source

Dataset Name: MNIST Handwritten Digit Dataset

Source:

The dataset is accessed using TensorFlow built-in API, which internally downloads data from Yann LeCun's MNIST database.

`tf.keras.datasets.mnist`

Or

Use this (easy format)

<https://www.kaggle.com/datasets/oddrational/mnist-in-csv>

2.Dataset Description

The MNIST dataset is a widely used benchmark dataset for image classification.

Dataset Details:

- Total Images: 70,000
- Training Images: 60,000
- Testing Images: 10,000
- Image Size: 28×28 pixels
- Channels: 1 (Grayscale)
- Number of Classes: 10 (digits 0–9)

Target Variable:

Digit label (0–9)

Data Representation:

- Each image is a **28×28 matrix of pixel values**
- Pixel values range from **0 to 255**
- Values are normalized to **0–1**

3.Mathematical Formulation of CNN

CNN consists of multiple layers:

Convolution Layer

$$Feature\ Map = (Input * Kernel) + Bias$$

- Kernel (filter) extracts features such as edges and shapes

Activation Function (ReLU)

$$f(x) = \max(0, x)$$

- Removes negative values
- Adds non-linearity

Pooling Layer (Max Pooling)

$$Output = \max(region)$$

- Reduces size of feature maps
- Retains important features

Fully Connected Layer

$$Z = W.X + b$$

Softmax Output

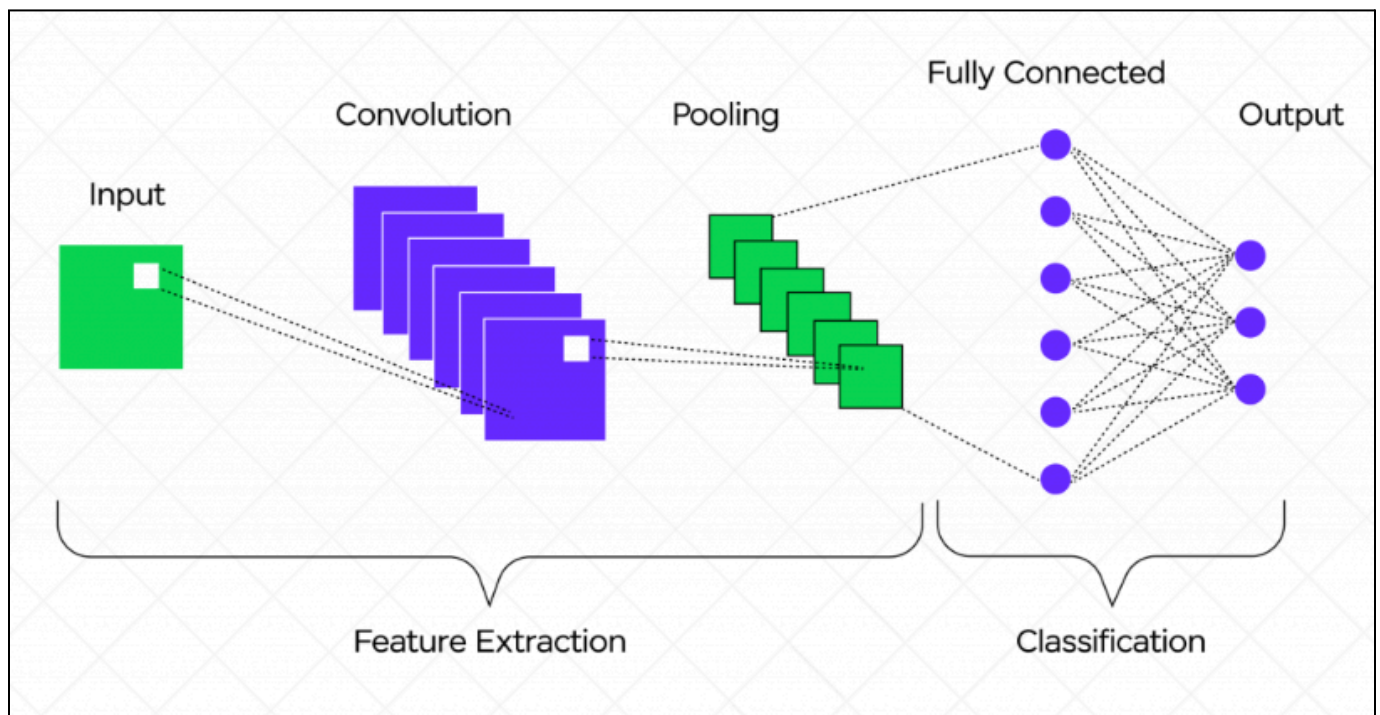
$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum e^{z_j}}$$

- Produces probability distribution

Loss Function

Categorical Crossentropy :

$$\text{Loss} = - \sum y \log(\hat{y})$$



4. Algorithm Limitations of CNN

Although Convolutional Neural Networks (CNNs) are highly effective for image processing tasks, they have several limitations:

1. High Computational Cost

CNNs require significant computational power because:

- Convolution operations involve heavy matrix calculations
- Deep networks consist of multiple layers

As a result:

- Training requires powerful hardware (GPU/TPU)
- Execution on normal systems can be slow

Example:

Face recognition systems in smartphones use GPUs to process images efficiently.

2. Requires Large Dataset

CNN models perform best when trained on large datasets.

If the dataset is small:

- The model cannot learn meaningful patterns
- Accuracy becomes low

Example:

Training a digit recognition model on only 20 images will result in poor predictions.

3. Overfitting Problem

CNN may memorize training data instead of learning general patterns.

Signs of overfitting:

- High training accuracy
- Low testing accuracy

Techniques to reduce overfitting:

- Dropout
- Data augmentation
- Regularization

Example:

The model performs well on training images but fails on new unseen images.

4. Complex Architecture Design

Designing a CNN is not straightforward.

- Choosing number of layers
- Selecting filter sizes
- Deciding number of neurons

Poor design leads to:

- Underfitting (too simple)
- Overfitting (too complex)

5. Time-Consuming Training

Training CNN models takes a significant amount of time due to:

- Large dataset size
- Multiple epochs
- Complex computations

Example:

Training medical image models may take hours or even days.

6. Sensitive to Hyperparameters

CNN performance depends heavily on hyperparameters such as:

- Learning rate
- Batch size
- Number of filters
- Epochs

Incorrect values may lead to:

- Slow learning
- Poor accuracy

Example:

A very high learning rate may cause unstable training.

7. Requires Data Preprocessing

Before training, images must be:

- Normalized (scaled between 0 and 1)
- Reshaped into proper format
- Converted into suitable input form

Raw data cannot be directly used.

5. Methodology / Workflow

The implementation of CNN follows a systematic step-by-step process:

Step 1: Dataset Loading

The MNIST dataset is loaded using TensorFlow

It provides:

- Training data (X_train, y_train)
- Testing data (X_test, y_test)

Step 2: Data Preprocessing

Before training, the dataset is prepared:

- **Normalization:** Pixel values are scaled from 0–255 to 0–1
- **Reshaping:** Images are converted into 4D format (samples, height, width, channels)
- **Encoding:** Labels are converted into categorical format using one-hot encoding

This improves model performance and training stability.

Step 3: Model Building

A Convolutional Neural Network is constructed using layers:

- **Convolution Layers:** Extract features

- **Activation (ReLU):** Introduces non-linearity
- **Pooling Layers:** Reduce dimensions
- **Flatten Layer:** Converts data to 1D
- **Dense Layer:** Performs classification
- **Output Layer (Softmax):** Produces probability of each class

Step 4: Model Compilation

The model is compiled using:

- **Optimizer:** Adam
- **Loss Function:** Categorical Crossentropy
- **Evaluation Metric:** Accuracy

Step 5: Model Training

The model is trained using training data:

- Multiple epochs are used
- Batch processing is applied
- Validation data is used to monitor performance

Step 6: Model Evaluation

The trained model is tested on unseen data:

- Accuracy is calculated
- Loss value is measured

Step 7: Visualization

Graphs are plotted:

- Accuracy vs Epochs
- Loss vs Epochs

These help in analyzing model learning behavior.

Workflow Summary

Images → Preprocessing → CNN Layers → Training → Evaluation → Results

6. Performance Analysis

The performance of the CNN model is evaluated using the following metrics:

1. Accuracy

- Measures how many predictions are correct
- Value ranges from 0 to 1

Higher accuracy = better model

2. Loss

- Measures error between predicted and actual values
- Lower loss = better performance

Observations

- Training accuracy increases gradually
- Validation accuracy remains close to training accuracy
- Loss decreases with epochs

This indicates:

- Model is learning properly
- No major overfitting

Result Interpretation

- High accuracy (~98–99%) shows strong model performance
- Low loss indicates minimal prediction error
- Model generalizes well to unseen data

7. Hyperparameter Tuning

Hyperparameters are values set before training that control model behavior.

Important Hyperparameters

Hyperparameter	Description
Epochs	Number of training cycles
Batch Size	Number of samples per update
Filters	Number of feature detectors
Kernel Size	Size of convolution filter
Learning Rate	Speed of learning
Dropout Rate	Prevents overfitting

Tuning Process

Different values are tested and compared:

Example:

- Epochs: 5, 10, 15
- Batch Size: 32, 64
- Dropout: 0.3, 0.5

Best combination is selected based on:

- Highest accuracy
- Lowest loss

Impact of Tuning

- Improves model accuracy
- Reduces overfitting
- Makes model stable
- Enhances generalization

Tools Used

- Manual tuning
- Trial and error

Conclusion

The CNN model was successfully trained and evaluated using the MNIST dataset. Proper preprocessing and hyperparameter tuning resulted in high accuracy and efficient performance in image classification tasks.